
PackageManifestInstaller

Sistema de gestión de dependencias UPM para plantillas Unity

Versión del schema	2.0
Organización	INVELON Technologies
Audiencia	Desarrolladores internos
Unity mínima	6000.3 (Unity 6)
Pipeline	Universal Render Pipeline (URP)

Índice de contenidos

- 1. Introducción y propósito**
- 2. Requisitos y compatibilidad**
- 3. Cómo abrir el instalador**
- 4. Auto-descubrimiento de manifests**
- 5. Interfaz del instalador**
 - 5.1 Cabecera y botón Refrescar
 - 5.2 Barra de pestañas
 - 5.3 Tabla de paquetes
- 6. Estados de paquetes**
- 7. Instalación de paquetes**
 - 7.1 Instalar paquete individual
 - 7.2 Install All
 - 7.3 Proceso de cola de instalación
- 8. Soporte de fuentes de paquetes**
- 9. Configuración automática de Scoped Registries**
- 10. Exportar snapshot de paquetes instalados**
- 11. Crear un nuevo archivo .dependencies.json**
- 12. Referencia completa del schema v2.0**
- 13. Ejemplo funcional: vr-template**
- 14. Errores comunes y soluciones**

1. Introducción y propósito

El **PackageManifestInstaller** es una ventana de editor de Unity que automatiza la instalación y verificación de paquetes UPM en proyectos de plantilla de INVELON. Elimina la necesidad de editar manualmente el archivo `Packages/manifest.json` y garantiza que cualquier desarrollador que abra una plantilla vea exactamente qué paquetes están instalados, cuáles faltan y con qué versión.

El sistema se compone de tres piezas:

- **PackageManifestInstaller.cs** — ventana de editor Unity (EditorWindow).
- **PackageManifest.schema.json** — esquema JSON v2.0 que define el formato de los manifests.
- ***.dependencies.json** — uno o más archivos de configuración, uno por plantilla, descubiertos automáticamente.

El script reside en su propio assembly definition (`INVELON_PackageManagement_Editor.asmdef`) sin referencias externas, de forma que compila incluso cuando otros assemblies tienen errores por paquetes ausentes.

2. Requisitos y compatibilidad

Unity	6000.3 o superior
Pipeline	Universal Render Pipeline (URP)
Assembly	INVELON_PackageManagement_Editor.asmdef sin dependencias externas
Schema	schemaVersion: "2.0" (versiones anteriores no son compatibles)
Ubicación del script	Cualquier carpeta dentro de Assets/ con acceso a Editor assemblies

3. Cómo abrir el instalador

Existen dos formas de acceder a la ventana:

Opción A — Menú principal de Unity

INVELON > Package Manager > Dependency Installer

Opción B — Menútem personalizado por plantilla

Si el archivo `*.dependencies.json` de una plantilla define el campo `menuGroup`, el instalador registra automáticamente un acceso directo adicional en ese path. Por ejemplo, para la plantilla VR:

INVELON > VR Template

La ventana tiene un tamaño mínimo de 560 × 460 px. Si se redimensiona por debajo de ese valor, Unity aplica el mínimo automáticamente.

4. Auto-descubrimiento de manifests

Al abrirse (o al pulsar **■ Refrescar**), el instalador ejecuta `AssetDatabase.FindAssets("t:TextAsset")` y filtra todos los archivos cuyo nombre termine en `.dependencies.json`. Cada archivo encontrado se convierte en una pestaña independiente en la ventana.

El proceso de carga de cada manifest sigue esta secuencia de validaciones:

- Lectura del archivo desde disco.
- Comprobación de que el JSON no está vacío.
- Deserialización con `JsonUtility.FromJson<TemplateManifest>`.
- Verificación de que `schemaVersion == "2.0"`.
- Verificación de que la lista `packages` no está vacía.

Si ningún archivo `*.dependencies.json` es encontrado, la ventana muestra un mensaje de error con un botón *Reintentar descubrimiento*. Asegúrate de que el XLINK de la plantilla está aplicado y que `AssetDatabase` ha refrescado.

Para añadir una nueva plantilla: crea un archivo `*.dependencies.json` en cualquier carpeta dentro de la plantilla XLINK'd. El instalador lo detectará automáticamente en el siguiente refresco sin necesidad de modificar el código.

5. Interfaz del instalador

5.1 Cabecera y botón Refrescar

La cabecera muestra el título **Dependency Installer** y un contador del tipo *N plantilla(s) detectada(s) · *.dependencies.json*. El botón **■ Refrescar** está deshabilitado mientras haya una operación de instalación o una `ListRequest` en curso.

5.2 Barra de pestañas

Cada manifest cargado aparece como una pestaña con el valor de su campo `templateName`. Si el manifest tiene un error de carga, la pestaña muestra el prefijo **■**. Hacer clic en una pestaña también hace *ping* al asset en el Project window.

5.3 Tabla de paquetes

Cada fila de la tabla muestra las siguientes columnas:

Fuente badge	Pill coloreado: REG (registry), GIT o NPM (openupm).
Paquete	<code>displayName</code> del paquete. El tooltip muestra el id UPM completo.
Requerida	Versión mínima definida en el manifest.
Instalada	Versión detectada en el proyecto, o — si no está instalado.
Estado	Badge de color (ver sección 6).
Acción	Botón Install o Update si el paquete no está al día; deshabilitado si ya lo está.

Si un paquete tiene `installNote` definido, se muestra una línea adicional en cursiva bajo la fila, con instrucciones de fallback manual.

6. Estados de paquetes

El instalador compara las versiones usando `System.Version` para paquetes de tipo `registry`. Para paquetes `git` y `openupm` solo se verifica la presencia (no hay comparación de versiones).

Installed	La versión instalada $\geq$ versión requerida.
VersionLow	Paquete presente pero con versión inferior a la requerida.
Missing	Paquete no encontrado en el proyecto.
Loading	Consulta en curso al UPM Client.
Error	Fallo al ejecutar la <code>ListRequest</code> o <code>AddRequest</code> .

Los sufijos pre-release como `-pre.1` o `-exp.1` se eliminan antes de la comparación semántica para evitar falsos positivos en versiones experimentales de Unity.

7. Instalación de paquetes

7.1 Instalar paquete individual

Pulsando el botón **Install** (o **Update**) en la fila de un paquete se encola ese paquete y se inicia inmediatamente la instalación. El identificador que se pasa a `Client.Add()` depende de la fuente:

- **registry**: `id@version` — ej. `com.unity.timeline@1.8.7`
- **git**: `url directa` — ej. URL del repositorio con tag opcional
- **openupm**: configura el scoped registry en `manifest.json` (ver sección 9) y luego instala como `id@version`

7.2 Install All

El botón **Install All** del footer encola todos los paquetes con estado *Missing* o *VersionLow* de la sección **Requeridos**. Los paquetes marcados como `optional: true` **no** se incluyen en esta operación.

7.3 Proceso de cola de instalación

La instalación es secuencial para respetar las restricciones del UPM Client:

1. Se añade el paquete a la cola `_installQueue`.
2. Se llama a `Client.Add(identifíer)` y se registra `PollAddRequest` en `EditorApplication.update`.
3. Cuando la request completa, si hay más paquetes en cola se inicia la siguiente.
4. Al terminar la cola se ejecuta un nuevo `StartListRequest()` para actualizar los estados.

Durante la instalación todos los botones están deshabilitados. No cierres la ventana ni cambies de escena hasta que la barra de progreso desaparezca.

8. Soporte de fuentes de paquetes

registry	Registros de Unity	Sí	No	Paquetes del Unity Registry oficial. Instalación directa.
git	URL de repositorio Git	No	No	Se pasa la URL completa a <code>Client.Add()</code> . Soporte de <code>#tag</code> o <code>#branch</code> al final de la URL.

openupm	OpenUPM scoped registry	Sí	Sí	Requiere configurar el scoped registry en Packages/manifest.json antes de instalar. El instalador lo gestiona automáticamente.

Ver = Comparación de versión semántica activa. Reg. = Requiere configurar scoped registry.

9. Configuración automática de Scoped Registries

Para paquetes con source: "openupm", el instalador modifica directamente el archivo Packages/manifest.json del proyecto antes de lanzar la request de instalación. El proceso es:

- Lee el contenido actual de Packages/manifest.json.
- Busca si ya existe una entrada en scopedRegistries con la misma URL.
- Si existe, añade el id del paquete al array scopes si no estaba ya.
- Si no existe, crea un nuevo bloque de registry con name, url y scopes.
- Si no existe la sección scopedRegistries, la crea antes del cierre del JSON raíz.
- Escribe el archivo y llama a AssetDatabase.Refresh().

Ejemplo de bloque generado para un paquete OpenUPM:

```
"scopedRegistries": [
  {
    "name": "package.openupm.com",
    "url": "https://package.openupm.com",
    "scopes": [
      "com.eflatun.scenereference"
    ]
  }
]
```

El instalador usa manipulación de texto en lugar de un parser JSON completo para evitar dependencias externas. Si el manifest.json tiene un formato no estándar o comentarios, puede fallar silenciosamente. En ese caso, añade el scoped registry manualmente.

10. Exportar snapshot de paquetes instalados

El botón **Export Snapshot** del footer genera un nuevo archivo exported-YYYY-MM-DD-HHmm.dependencies.json en la misma carpeta del manifest activo. Este archivo refleja todos los paquetes instalados en el proyecto en ese momento.

Detalles del proceso de exportación:

- Ejecuta `Client.List(offlineMode: false, includeIndirectDependencies: false)`.
- Filtra paquetes del módulo `com.unity.modules.*` (módulos internos de Unity).
- Detecta paquetes **git** por `PackageSource.Git`.
- Detecta paquetes **openupm** inspeccionando el `manifest.json` en busca de scoped registries con URL que contenga *openupm*.
- El resto se marca como **registry**.

El snapshot exportado se auto-descubre en el siguiente refresco y aparece como una nueva pestaña. Es útil para documentar el estado de dependencias de un proyecto en un momento determinado o como punto de partida para una nueva plantilla.

11. Crear un nuevo archivo `.dependencies.json`

Pasos para añadir soporte a una nueva plantilla:

1. Crea un archivo con el nombre `mi-plantilla.dependencies.json` dentro de la carpeta `XLINK'd` de la plantilla (o en cualquier carpeta de `Assets`).
2. Asegúrate de que Unity lo indexa como `TextAsset` (debe estar dentro de `Assets/` y no en una carpeta excluida).
3. Rellena el archivo siguiendo el schema v2.0 (ver sección 12).
4. Abre el instalador o pulsa **■ Refrescar**. El archivo aparecerá como una nueva pestaña.

Esqueleto mínimo de un archivo válido:

```
{
  "schemaVersion": "2.0",
  "templateName": "Mi Plantilla",
  "packages": [
    {
      "id": "com.unity.timeline",
      "version": "1.8.7",
      "source": "registry",
      "displayName": "Timeline"
    }
  ]
}
```

12. Referencia completa del schema v2.0

Campos raíz del manifest

schemaVersion	string	✓	Siempre "2.0". Versiones distintas provocan error de carga.
templateName	string	✓	Nombre mostrado en la pestaña del instalador.
unityVersion	string	—	Versión mínima de Unity. Solo informativo.
renderPipeline	string	—	Pipeline requerida. Solo informativo. Ej: "URP".
menuGroup	string	—	Path para MenuItem extra. Ej: "INVELON/VR Template". Vacío para omitir.
packages	array	✓	Lista de entradas de paquete (ver tabla siguiente).

Campos de cada entrada en packages[]

id	string	✓	Identificador UPM. Ej: com.unity.xr.openxr
version	string	✓	Versión mínima requerida (semver). Para git es solo informativo.
source	string	✓	"registry" "git" "openupm"
url	string	■	Requerido para git (URL completa + #tag opcional) y openupm (siempre https://package.openupm.com).
scopeName	string	—	Solo openupm. Nombre visible del scoped registry en manifest.json. Por defecto usa el valor de url.
displayName	string	✓	Nombre legible mostrado en la tabla del instalador.
optional	bool	—	Si true, el paquete aparece en sección Opcionales y se excluye de Install All. Default: false.
installNote	string	—	Nota mostrada bajo la fila. Útil para instrucciones de fallback manual.

✓ Requerido ■ Condicional según source — Opcional

13. Ejemplo funcional: vr-template.dependencies.json

A continuación se muestra el manifest completo de la plantilla VR (*InvelonARVsion VR Template*), que incluye paquetes de los tres tipos de fuente:

```
{
  "schemaVersion": "2.0",
  "templateName": "InvelonARVsion VR Template",
  "unityVersion": "6000.3",
  "renderPipeline": "URP",
  "menuGroup": "INVELON/VR Template",
  "packages": [
    { "id": "com.unity.xr.openxr", "version": "1.16.1", "source": "registry", "displayNam
    { "id": "com.unity.xr.management", "version": "4.4.0", "source": "registry", "displayNam
    { "id": "com.unity.localization", "version": "1.5.9", "source": "registry", "displayNa
    { "id": "com.unity.inputsystem", "version": "1.13.1", "source": "registry", "displayNa
    { "id": "com.unity.render-pipelines.universal", "version": "17.0.3", "source": "registry", "displayNa
    { "id": "com.unity.timeline", "version": "1.8.7", "source": "registry", "displayNa
    {
      "id": "com.unity.xr.openxr.picoxr",
      "version": "1.4.1",
      "source": "git",
      "url": "https://github.com/Pico-Developer/PICO-Unity-OpenXR-SDK.git",
      "displayName": "PICO OpenXR Plugin",
      "installNote": "Si la instalacion automatica falla, instala manualmente desde Package Manager."
    },
    {
      "id": "com.eflatun.scenereference",
      "version": "5.0.0",
      "source": "openupm",
      "url": "https://package.openupm.com",
      "scopeName": "package.openupm.com",
      "displayName": "Eflatun.SceneReference"
    }
  ]
}
```

14. Errores comunes y soluciones

No se encontraron archivos *.dependencies.json

El XLINK de la plantilla no está aplicado, o AssetDatabase no ha indexado el archivo. Verifica con Assets > Refresh y comprueba que el archivo está dentro de Assets/.

Schema version 'X' no compatible

El campo schemaVersion no es "2.0". Actualiza el valor en el archivo .dependencies.json.

No se pudo deserializar [archivo]

El JSON tiene un error de sintaxis. Valida el archivo con un linter JSON antes de guardarlo.

Paquete git queda en estado Missing tras instalación

La URL puede ser incorrecta o el repositorio privado requiere autenticación SSH. Instala manualmente desde *Package Manager > Add package from git URL* para ver el error completo.

Scoped registry no se crea correctamente

El `manifest.json` puede tener un formato no estándar. Añade el scoped registry manualmente en `Packages/manifest.json` y vuelve a intentar la instalación.

Export Snapshot genera un archivo vacío

La `ListRequest` falló. Comprueba la consola de Unity en busca de errores del UPM Client y asegúrate de tener conexión de red activa.
